Model Comparison in Breast Cancer Classification: A Machine Learning Project

Matthew Blong

Dr. Yunpeng Zhao

STAA 578

May 16th, 2025

Introduction and Dataset Description

The University of Wisconsin developed a computerized method to diagnose breast cancer to a high degree of accuracy. A graphical interface, first, digitizes breast FNA's, or Fine Needle Aspirates, which is a minimally invasive biopsy where "a very thin needle is used to collect a sample of cells, tissue or fluid from an abnormal area or lump." The results are 640 × 400, 8-bit-per-pixel gray scale images. Then, a program called Xcyt analyzes each image to determine its shape and boundary, through which ten features are extracted. These features include:

a) radius (mean of distances from center to points on the perimeter)
b) texture (standard deviation of gray-scale values)
c) perimeter
d) area
e) smoothness (local variation in radius lengths)
f) compactness (perimeter^2 / area - 1.0)
g) concavity (severity of concave portions of the contour)
h) concave points (number of concave portions of the contour)
i) symmetry
j) fractal dimension ("coastline approximation" - 1)

For each of these cellular features, the mean value, extreme value, and standard error are calculated, for a total of 30-dimensional points for each digitized breast FNA sample. Each sample is then labeled as either "Malignant" or "Benign". As a result, 569 samples (357 of which are labeled as "Benign" and 212 "Malignant") were collected from January 1989 to November 1991 and compiled into the Breast Cancer Wisconsin Diagnostic Data Set, which is publicly available at the UC Irvine Machine Learning Repository.

This data set has contributed to developing methods to better predict malignancy in breast cancer. Early detection of breast cancer is crucial to patients so that it can ensure improved survival rates, less aggressive treatment options, and better quality of life. In this report, we are going use Python to compare several machine learning models (deep neural network, logistic regression, support machine vector, random forests and XGBoost) to demonstrate which model is best suited for this classification.

Data Preprocessing

The data set was loaded using the "ucimlrepo" module in Python and separated into their respective feature and target values. There were no missing values. We, first, encoded our target values from "M" (malignant) to 1 and "B" (benign) to 0. Second, we normalized our features with the StandardScaler function. And third, we split our newly encoded and scaled data into training and test sets with the train_test_split function with a test size of 0.2.

Method

We started our analysis by building a simple neural network with two hidden layers, one with 64 nodes and the other with 32, each with a non-linear Relu activation. The output, since this is a

binary classification, has one neuron and a sigmoid activation as the model is predicting the probability that the output belongs to class 1. We also included dropout layers as a form of regularization to prevent overfitting. We then compiled our model with an Adam optimizer with a learning rate of 0.001. We use an Adaptive Moment Estimation optimization algorithm due to its ability to reach convergence faster than SGD and RMSprop, accelerate learning, deal with sparse data, and handle default settings (such as our 0.001 learning rate). Our loss function was binary cross-entropy because it compares probabilities, the output of our model, with the actual binary labels and penalizes based on the distance between the former and the latter values. We fit our model with our training data with mini-batch gradient descent to strike a balance between speed and stability. We used a batch size of 16 since we had limited memory, and to which the noise of the updates may help avoid overfitting, and 50 epochs to give the model enough time to properly learn the data.

Next, we created a baseline model to compare the results from our neural network. We chose a logistic regression as our baseline, which consisted of a single dense layer with one neuron and a sigmoid activation. Our baseline included all the same optimizers, loss functions, and fit criteria as stated above.

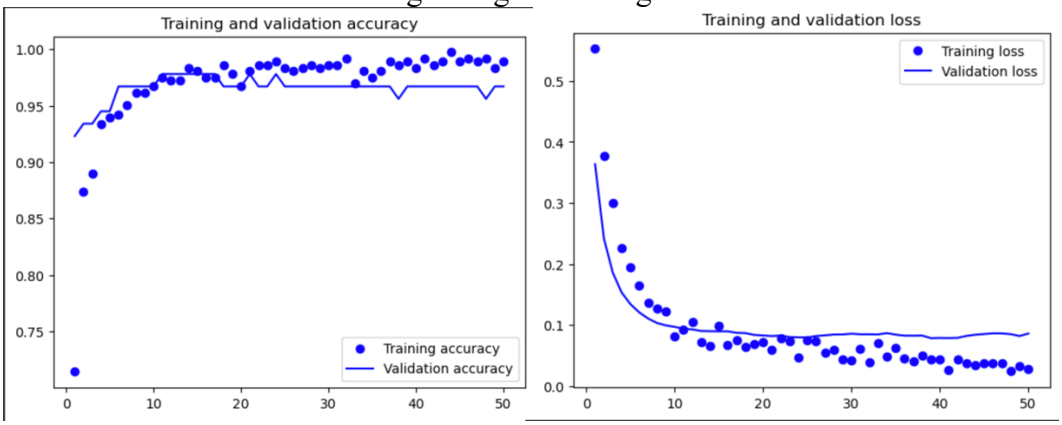
We compare these models to other machine learning methods to find the best predictive model. First, we start with a Random Forest Classifier, which is an ensemble learning algorithm that builds multiple decision trees during training. Each tree is trained on a bootstrap sample (randomly sampled with replacement) of the original dataset, using a random subset of features at each split to reduce correlation between trees. This approach improves generalization by reducing overfitting and increasing predictive accuracy. Each tree makes its own prediction and the class that most of the trees choose is the final prediction for each sample. Although trees grow independently without pruning, majority voting helps reduce overfitting. Our initial model is a Random Forest with 100 trees.

The next model we evaluate is the Support Vector Machine (SVM). SVMs work by finding the best possible boundary—called a hyperplane—that separates the data into classes. In our case, since we are dealing with a binary classification problem, the SVM finds the optimal line (or surface in higher dimensions) that maximally separates the two classes. It does this by focusing on the data points that are closest to the boundary, known as support vectors, to ensure the widest possible margin between the classes. Important hyperparameters for SVM are kernel (a function that maps the data to higher-dimensional space), cost (a regularization term), and gamma (controls the shape and complexity of the decision boundary). Our initial SVM model uses a radial basis function kernel, which transforms data based on the distance between points; a cost function of 1, which is default for many SVMs; and a gamma is set to ‘scale’ as to dynamically adjust gamma based on the spread of the data.

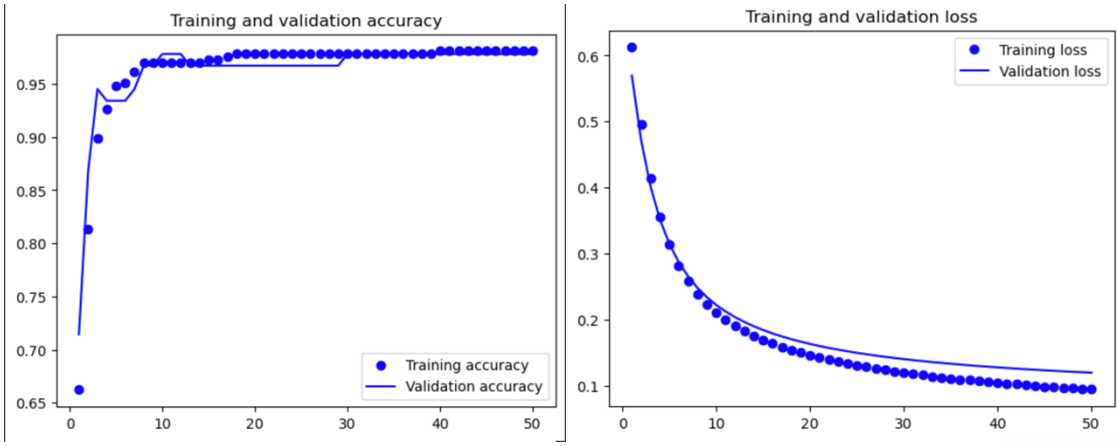
Our final model that we explore is XGBoost, another ensemble learning method that optimizes gradient boosting, which sequentially combines decision trees to improve model performance by correcting errors made by previous trees. The final prediction, after a stopping criterion is met, is the sum of all its decision trees’ predictions. Its objective function consists of a loss function, which measures how well the model fits the data, and a regularization term, which penalizes complex trees. Then, information gain is calculated to determine which splits result in the largest gain to reduce error and improve performance. Our initial model uses a log loss function, which penalizes high probabilities for incorrect predictions and low probabilities for correct predictions.

Outcome

For our deep neural network, we achieved a test accuracy of 0.9737. Because the data set is skewed (substantially more benign cases than there are malignant cases), we also calculated F1-score, which is a combination of recall and precision, since it gives a balanced view of false positives and false negatives for imbalanced data sets. Our F1-score was computed as 0.965. As we can see from the plots below, our training and validation accuracy track together and sharply increase around 5 epochs and then keeps steadily increasing from then on, almost plateauing, which illustrates that our model is doing well generalizing the data.



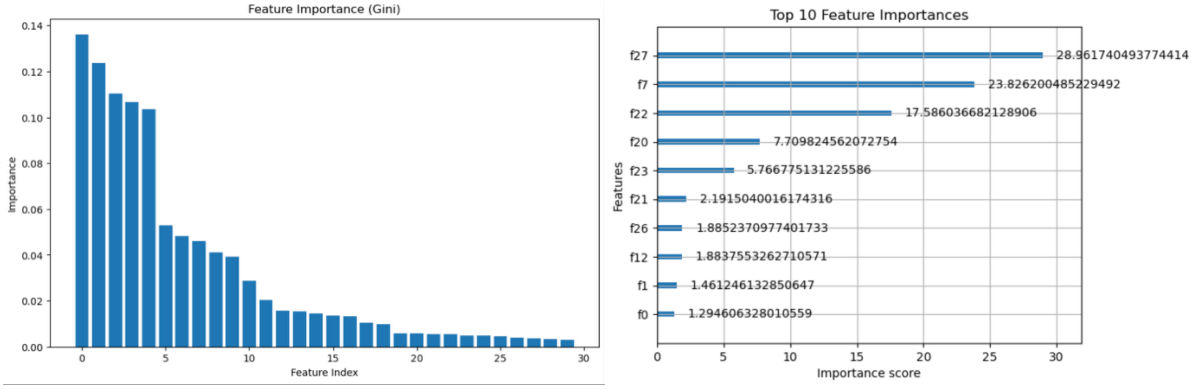
Our baseline logistic regression achieved the same test accuracy of 0.9737 (and the same F1-score: 0.965). We see, as well, how our training and validation accuracy track together with the same sharp increase and plateau as our deep neural network. However, this sharp of an increase may suggest that the model is merely memorizing the training data by overfitting rather than truly generalizing.



For our Random Forests, using 100 trees, we fit a model that has an overall accuracy of 0.965 and the weighted F1-score is also 0.96. Our initial SVM, we have an over accuracy of 0.97 and a weighted F1-score of 0.97 as well. Lastly, our initial XGBoost model resulted in an overall accuracy of 0.956 and a weighted F1-score of 0.96.

Random Forest and XGBoost models also inherently track feature importance when fitting our training data. In the graphs below, we see how the two models rank features by their

importances. Random Forest calculate importance as the average total reduction in impurity, which is whenever a feature is used to split a node. On the left, we see that the first five features have the greatest importance and the biggest contributors to reducing uncertainty in our model. In contrast, we XGBoost measures importance through average gain, which is how much a feature improves the split quality across all trees. On the right, we get a different combination of important features, where the 27th, 7th, and 22nd features are the three most important values to increase accuracy across all trees on average.



Hyperparameter Tuning

In order to efficiently fine tune our neural network, we used Keras Tuner. First, we build a general function that resembled our previous model but our hyperparameters ranged from set minimum and maximum values to properly define our search space. The Hyperband function then randomly samples large number of configurations and evaluates each of these configurations with a small number of epochs. After this initial process, Hyperband then prunes the worst-performing configurations so that it can allocate more resources to better-performing configurations. This process continues until Hyperband finds the hyperparameters with the best performance. After running the function, we found that best hyperparameters for our model:

- Input Units: 96
- Hidden Units: 96
- Dropout Input: 0.4
- Dropout Hidden: 0.1
- Learning Rate: 0.0013

Our test accuracy for the fine-tuned model is 0.965. This slightly lower accuracy compared to our original DNN may be due to overfitting caused by increasing the number of neurons in the hidden layers. While more complex architectures can capture intricate patterns, they also risk learning noise in the training data. In this case, the original, simpler DNN model appears to generalize better to the test set, making it a better fit for our dataset.

We fine tune our Random Forest model using RandomizedSearchCV, which fine tunes the number of trees, maximum depth of each tree, minimum number of samples required to split an internal node, minimum number of samples required to be at a leaf node, and the number of features when looking for the best split. For each combination, RandomizedSearchCV evaluates each Random Forest with cross-validation to find the best-performing set of hyperparameters:

- n_estimators: 1000
- min_samples_split: 2
- min_samples_leaf: 2
- max_features: 'sqrt'
- max_depth: None

However, the fine-tuned hyperparameters once again produced an accuracy score of 0.965. This may be due to the fact that Random Forests are relatively robust out-of-the-box and often perform well with default settings, requiring less hyperparameter tuning compared to other models like Support Vector Machines or neural networks.

We fine tune the SVM model with RandomizedSearchCV. We build a function to properly tune the hyperparameters of interest. We separate them by kernel ('linear' or 'rbf'), make sure that all features are properly scaled with a pipeline, use a balanced class weight since our data set is imbalanced, use a log uniform distribution to search across our C and gamma, and used a stratified k-fold for a more reliable validation. Our best hyperparameters are

- C: 0.0107
- Gamma: 0.00117
- Kernel: linear

These hyper parameters produced an accuracy of 0.991 and a weighted F1-score of 0.99.

We, finally, fine tune our XGBoost model. We perform hyperparameter tuning using RandomizedSearchCV. It defines a search space for key parameters such as the number of trees (n_estimators), tree depth (max_depth), learning rate, and sampling ratios. The search randomly evaluates 50 different combinations using 3-fold cross-validation, optimizing for the lowest mean squared error. Once the best set of parameters is found, the model is trained on the training data and used to make predictions on the test set. Our best hyperparameters are

- colsample_bytree: 0.635
- learning_rate: 0.162
- max_depth: 9
- min_child_weight: 4
- n_estimators: 126
- subsample: 0.813

This fine-tuned XGBoost model produced a test accuracy of 0.974 and a weighted F1-score of 0.97, which is marginally better than our original model.

Conclusion

Through our exploration of different machine learning models, we found that, after fine tuning our hyper parameters, our Support Machine Vector model produced the highest test accuracy and F1-score, which is the harmonic mean of precision and recall, for predicting malignant versus benign tumors from the Breast Cancer Wisconsin Diagnostic Data Set. Using a Support Machine Vector for this type of task could lead to improving health outcomes for those suspected of breast cancer for future data points.

References

GeeksforGeeks. 2021. “XGBoost.” GeeksforGeeks. September 18, 2021. https://www.geeksforgeeks.org/xgboost/.

Lee, Sid. 2015. “Fine Needle Aspiration (FNA).” Canadian Cancer Society. 2015. https://cancer.ca/en/treatments/tests-and-procedures/fine-needle-aspiration-fna.

Mangasarian, Olvi L., W. Nick Street, and William H. Wolberg. 1995. “Breast Cancer Diagnosis and Prognosis via Linear Programming.” *Operations Research* 43 (4): 570–77. https://doi.org/10.1287/opre.43.4.570.

McGrath, Eoghan. 2024. “Why Is Early Detection of Breast Cancer Important - Ezra.” Ezra.com. June 18, 2024. https://ezra.com/blog/why-is-early-detection-of-breast-cancer-important.

Sasidharan, Aswathi. 2021. “Support Vector Machine Algorithm.” GeeksforGeeks. January 20, 2021. https://www.geeksforgeeks.org/support-vector-machine-algorithm/.

Wolberg, William, Olvi Mangasarian, Nick Street, and W. Street. 1995. “UCI Machine Learning Repository.” Archive.ics.uci.edu. October 31, 1995. https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic.